WhatTheBird - Klassifikation von deutschen Vögeln mit CNNs

Niklas Zöscher, Marcus Gaidamak

9. Februar 2025

Abstract

Der Bericht behandelt die Entwicklung eines Convolutional Neural Networks (CNN), das darauf trainiert wird, anhand eines selbst erstellten Datensatzes, die Vogelart von Testbildern vorherzusagen. Der Bericht kommt zu dem Ergebnis, dass der Einsatz von Transfer Learning (VGG16) und Data Augmentation zu hervorragenden Ergebnissen bei Testdaten aus dem ursprünglichen Datensatz führt. Jedoch bei Verwendung von externen Bildern, die Genauigkeit sinkt.

1 Einleitung

Die automatische Erkennung und Klassifikation unterschiedlicher Vogelarten anhand von Bildern ist eine anspruchsvolle und komplexe Aufgabe des maschinellen Lernens. Jedoch kann ein solches System wichtige Aufgaben in verschiedenen Anwendungsbereichen übernehmen. Von der wissenschaftlichen Forschung bis zum Hobby Ornithologen[1]. Unsere Domäne zeichnet sich dadurch aus, dass wir uns hauptsächlich mit der Erkennung von heimischen Wintervögeln auseinandergesetzt haben. Unsere konkrete Aufgabe bestand darin ein Modell zu trainieren, welches anhand eines Datensatzes, verschiedenste Vögel richtig kategorisiert. Unser Ziel ist es, das Modell später mit eigenen Fotos zu testen und eine vernünftige Trefferquote zu erzielen. Der Datensatz bestand aus 7 Klassen, welche bewusst gewählt wurden, um die am häufigsten vorkommenden Vögel in Deutschland abzudecken. Da die Exemplare durch ihre ähnliche Physiognomie teils nur schwer zu unterscheiden waren, stellt die Aufgabenstellung eine besondere Herausforderung dar. Damit man eine richtige Zuordnung zu einer hohen Wahrscheinlichkeit garantieren kann, bedarf es einer feinen Extraktion von Merkmalen. Um die Komplexität trotzdem zu bewältigen, haben wir uns für den Einsatz eines Convolutional Layers (CNN) entschieden. Im Folgenden gehen wir näher auf unsere Vorgehensweise und die dadurch entstandenen Ergebnissen ein.

[1] Ein Ornithologe ist ein Wissenschaftler, der sich mit Vögeln beschäftigt

2 Datensatz

Ein zentraler Bestandteil jedes maschinellen Lernmodells ist der zugrunde liegende Datensatz, da er maßgeblich die Qualität und Leistungsfähigkeit bestimmt. Auch unser Datensatz bildet hier keine Ausnahme und spielt eine bedeutende Rolle für die Genauigkeit des Modells. Innerhalb unserer Erarbeitung sind wir auf einige Vogel-Datensätze gestoßen, allerdings enthielten die Datensätze selten Vögel, die wir mit unserem Modell abdecken wollten. Konkret haben einige Datensätze, Flamingos mit Eulen verglichen und passten im Konsens nicht zu den deutschen oder europäischen Wintervögeln, die wir beobachten wollen.

2.1 Datenerhebung

Für die Datenerhebung und Erstellung eines geeigneten Datensatzes haben wir die Bilder aus der Macauly Library[2] verwendet. Die Macauly Library ist ein Web-Archiv mit dem Schwerpunkt auf Vogel und Amphibien Aufnahmen und wird für wissenschaftliche Arbeiten, Lehrzwecke und auch für KI-Trainings verwendet. In diesem spezialisierten Archiv sind die Bilder geprüft und häufig professionell aufgenommen. Das Archiv ermöglicht uns einen Zugang auf über 50.000 Amsel Aufnahmen, die gut geeignet sind für Trainingsdaten. Die letztendliche Erhebung der Daten ist durch einen Sitescraper realisiert worden, der für uns automatisch die Bilder herunterlädt und haben ihn mittels des Selenium[3] Frameworks aufgebaut.

2.2 Inhalt

Unser Datensatz WhatTheBird25 (WTB25) beinhaltet 500 Bilder pro Klasse. Summa summarum umfasst WTB25 7 Klassen, was zu einer Gesamtzahl von 3500 Bildern führt. Diese wurden unterteilt in 100 Test und 400 Trainingsbilder. Des Weiteren wurden zufällige 20 % der Trainingsdaten als Validierungsbilder verwendet. Die Klassen bestehen aus unterschiedlichen deutschen Vögeln, die sich durch einzigartige Merkmale unterscheiden. Das bedeutet jedoch nicht, dass alle Arten leicht zu unterscheiden sind. Wie in Abb. 3 zu sehen, haben wir bewusst Arten gewählt, die sich gewissermaßen ähneln, um die Robustheit des Systems zu testen.

[2]The Selenium Browser Automation. Retrieved February 2, 2025, from https://www.macaulaylibrary.org/about/

[3]Macaulay Library. (n.d.). About. The Cornell Lab of Ornithology. Retrieved February 8, 2025, from https://www.selenium.dev/documentation/



Abb. 1: Kohlmeise



Abb. 2: Blaumeise

Abb. 3: Vom Äußerlichen sind Kohl und Blaumeise nur durch ihre Farben zu unterscheiden. Form und Muster sind weitgehend gleich

2.3 Qualität der Bilder

Da die Beschaffung des Datensatzes automatisiert erfolgte, war es nicht möglich, einen einwandfreien Datensatz zu erschaffen. In den Klassen befanden sich teils Bilder, die nicht zu einem positiven Ausgang beitrugen, wie zum Beispiel Fotos, die unscharf waren oder aus hoher Entfernung aufgenommen wurden. Jede Klasse wurde jedoch grob kontrolliert, um die Majorität an minderwertigen Fotos auszusortieren. Der Löwenanteil der benutzten Bilder ist jedoch von hoher Qualität und damit geeignet, maschinelles Lernen zu ermöglichen.



Abb. 4: Rotkehlchen



Abb. 5: Kohlmeise



Abb. 6: Amsel



Abb. 7: Blaumeise



Abb. 8: Buchfink



Abb. 9: Feldsperling



Abb. 10: Erlenzeisig

Obwohl die Fotos allgemein von hoher Qualität waren, gab es zwischen den Klassen deutliche Unterschiede. So umfassen die Klassen der Abb. 4-7 hauptsächlich Bilder, die scharf und gut belichtet waren. Wodurch die Erkennung von Formen und Farben leichter fiel. Dagegen weisen Abb. 8-10 eine höhere Varianz an Bilderqualitäten auf. Hier kommt es häufiger vor, wie in Abb 13 zu sehen ist, dass Bilder aus großer Entfernung oder nicht idealen Perspektiven aufgenommen wurden.



Abb. 11: Erlenzeisig im Flug



Abb. 12: Feldsperlinge im Busch

Abb. 13: Zwei Beispiele für minderwertige Bilder

3 Methodik

Aufgrund der unterschiedlichen Beschaffenheit der Bilder haben wir uns gegen eine klassische Extraktion von Merkmalen und für eine Nutzung eines CNNs entschieden.

3.1 Verarbeitung der Daten

Da wir mit nur 500 Bildern pro Klasse einen eher kleinen Datensatz hatten, mussten wir die Trainingsdaten künstlich vermehren, damit das Modell auch Bilder von schlechter Qualität richtig zuordnen und allgemein besser generalisieren kann. Dies haben wir mit Data Augmentation umgesetzt. Es war erstmals entscheidend, alle Bilder auf eine einheitliche Größe zu bringen, damit das CNN arbeiten kann. Aufgrund unserer Entscheidung VGG 16 zu verwenden wurden alle Bilder auf das Format 224x224 gebracht, da dies der Standard von VGG 16 ist und aber auch gleichzeitig sichergestellt werden kann, dass die Bilder groß genug sind, um brauchbare Merkmale extrahieren zu können. Zusätzlich wurden Augmentationen benutzt, die in Abb. 14 veranschaulicht sind.
Es war entscheidend, darauf zu achten, dass nicht zu viele Augmentationen gleichzeitig angewendet werden, um zu verhindern, dass das Bild zu stark verfälscht wird und dadurch wichtige Merkmale verloren gehen. Aus diesem Grund wurden wenige Augmentationen gewählt, mit einem kleinen Veränderungsfaktor.

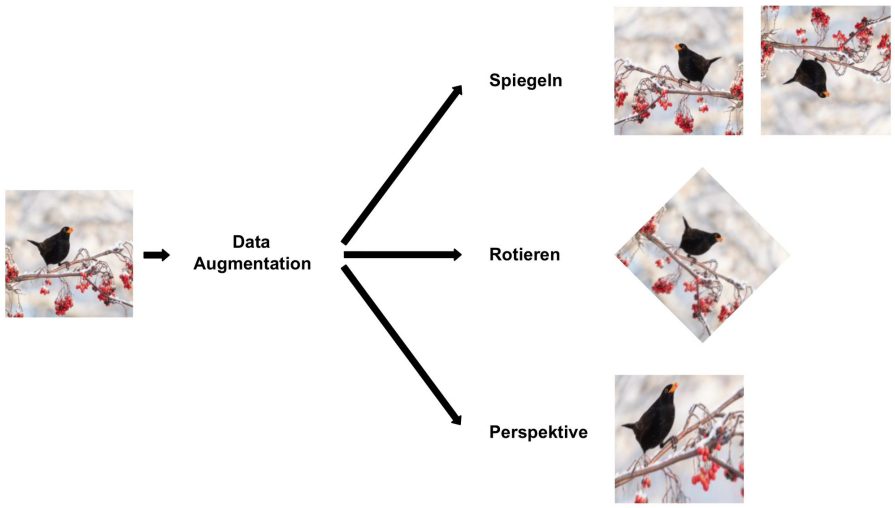


Abb. 14: Grafik für Augmentationen

Augmentationen, die Farben oder Muster verändern, können die Trefferquote deutlich verschlechtern oder verbessern, daher wurde lediglich die Helligkeit angepasst und vereinzelt der Farbton minimal verändert. Mit diesen Augmentationen sollen die Grundmerkmale des Vogels erfüllt bleiben und lediglich schwierige Bildbedingungen emulieren. Im nächsten Abschnitt werden wir detaillierter auf das verwendete Netz eingehen.

4 Aufbau des Netzes

Das verwendete Netz basiert auf dem VGG16-Netz, welches bereits auf 23 verschiedene Vogelarten trainiert wurde. Die Parameter dieses vortrainierten Netzes wurden eingefroren, um den Großteil des Trainings zu sparen. Nach diesem Basisnetz folgen zwei zusätzliche Convolutional Layer, die die 512 Merkmalkarten weiterverarbeiten und dienen der zusätzlichen Feinabstimmung. Als Aktivierungsfunktion wird ReLU (Rectified Linear Unit) eingesetzt. Anschließend reduziert eine Flatten-Schicht die Parameteranzahl, bevor der finale Wert über insgesamt drei Fully Connected Layer berechnet wird. Die genaue Konstruktion des Netzes haben wir über Veranstaltungsfolien und probierendes Vorgehen festgelegt.

4.1 Batch-Normalisierung

Leider waren die ersten Iterationen des Modells mit Zunahme der Klassen unzuverlässig und hatte lediglich eine Trefferquote von 50 %. Ein Problem in der Verwendung von ReLU und unserer Bilddaten war, dass negative Kantengewich-

te ignoriert wurden und dadurch manche Neuronen nur langsam oder gar nicht gelernt haben. Der Trainingsfortschritt pro Epoche war dadurch nur gering und hat mittelmäßige Ergebnisse geliefert. Um diesem Problem entgegenzuwirken, wird eine Batch-Normalisierung[4] eingesetzt. Dabei werden die Gewichte in Relation zueinander gebracht werden, indem sie ihren Durchschnittswert und ihre relative Distanz zueinander beibehalten, aber gleichzeitig an einem konzentrierten Punkt gesammelt werden.
Durch die Batch-Normalisierung erhält ReLU eine bessere Grundlage, um die Merkmale sinnvoll zu trennen. Da die Gewichte an einem fokussierten Punkt zusammengeführt werden, kann die Aktivierungsfunktion effizienter lernen und die Punkte besser voneinander unterscheiden. Dies führt zu einer geringeren Anzahl Lernepochen und besseren Ergebnissen. Konkret verbessert sich dabei die Generalisierungsfähigkeit des Modells, da eine gleichmäßige Gewichtsverteilung ein robusteres Verhalten auf unbekannten Daten ermöglicht und damit Overfitting vorbeugt. Insgesamt trägt die Batch-Normalisierung zu einer besseren Performanz des Netzes bei, indem sie sowohl die Trainingsgeschwindigkeit als auch die Klassifikationsgenauigkeit erhöht.

5 Ergebnisse

Im folgenden Kapitel behandeln wir die Leistung des Netzes und die daraus resultierenden Ergebnisse. Ziel war es ein Modell zu entwickeln, dass sowohl bei den Testdaten als auch bei unseren eigenen Fotos (Predictdaten), eine hohe Trefferquote erzielt. Wir gehen auf Stärken, sowie auch auf Probleme ein und klären, wie man diese lösen kann.

5.1 Analyse der Ergebnisse

Unser bestes System kommt auf eine Treffergenauigkeit von 74-77 % bei 4 Iterationen. Hierbei wurden die in Kapitel 3.1 gezeigten Augmentationen bei einer Batchgröße von 16 durchgeführt. Das System hatte 20 Epochen Zeit, sich zu verbessern. Kam es jedoch nach fünf Epochen zu keiner Verbesserung, wurde der Durchlauf durch das Early Stopping abgebrochen. In der Regel hat das Modell neun Epochen gebraucht.

[4] PyTorch Documentation. Retrieved February 8, 2025, from https://pytorch.org/docs/stable/generated/torch.nn.BatchNorm1d.html

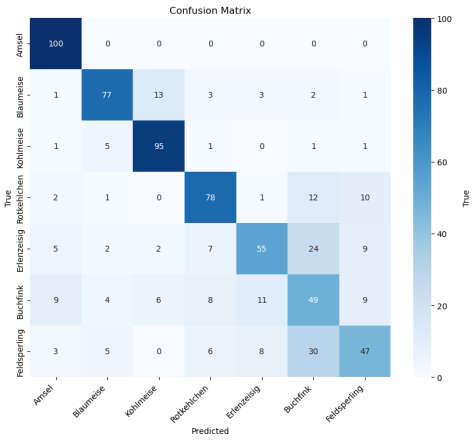


Abb. 15: Iteration 1

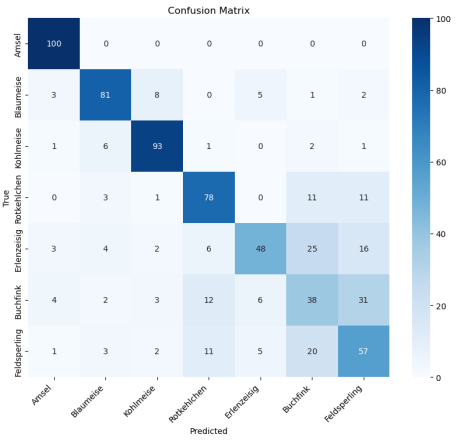


Abb. 16: Iteration 2

Abb. 17: Confusion Matrizen.

5.1.1 Gute Ergebnisse

In Abb. 17 lässt sich sofort erkennen, dass einige Vogelarten besser abschnitten als andere. Die Amsel schnitt mit 100 % am besten ab. Man könnte denken, durch ihr äußeres sei sie leicht von anderen zu unterscheiden. Das stimmt auch zum Teil. Jedoch befanden sich in unserem Amseldatensatz nicht nur männliche Exemplare. Wir haben bewusst auch weibliche eingefügt, welche sich schon deutlich vom männlichen Vogel unterscheiden. So ist die weibliche Amsel kleiner und hat ein braunes Gefieder, während die männliche tiefschwarz und größer ist. Man könnte sogar meinen, dass es sich hierbei um zwei unterschiedliche Arten handelt. Das System hat es trotzdem geschafft, die Geschlechter demselben Vogel zuzuordnen. Auch bei unserem Anfangs genannten Vögeln, der Blau- und Kohlmeise, wurde eine hohe Trefferquote erzielt. Obwohl die Blaumeise in einigen wenigen Fällen fälschlicherweise der Kohlmeise zugeordnet wurde, schafft es das System diese äußerlich sehr ähnlichen Vögel auseinanderzuhalten. Trotz der hohen Trefferquoten bei just diesen Arten gab es auch Vögel, bei denen das System eher Schwächen aufwies.

5.1.2 Schlechte Ergebnisse

So zum Beispiel beim Feldsperling, Erlenzeisig und Buchfink. Obwohl die Physiognomie dieser Vögel einen deutlich erkennbaren Unterschied aufweist, war das System nicht in der Lage, sie zuverlässig zuzuordnen. Obwohl das Äußerliche keine großen Ähnlichkeiten aufweist, beschränken sich die Verwechslungen hauptsächlich auf diese drei Vögel. Das ist ein Hinweis für ein Problem bei just diesen Klassen. Ein großer Aspekt ist die Qualität der Bilder. Wie bereits in Punkt 2.3 beschrieben, wies die Bildqualität in diesen drei Klassen die höchste Varianz auf. Die

Kombination aus einer hohen Trefferquote bei ähnlichen Vögeln und einer niedrigen bei nicht ähnlichen Vögeln lässt darauf schließen, dass die Bildqualität das Problem darstellt.

5.2 Predictergebnisse

Wie einleitend beschrieben, war es uns wichtig, dass das System auch auf neue, externe Testdaten zuverlässig reagiert. Deshalb haben wir unter anderem eigene Bilder verwendet, wie in Abb. 20 zu sehen, die in ihrer Qualität an Amateuraufnahmen erinnern. Für jede Klasse stellten wir dabei drei Fotos zur Verfügung, davon auch Amateuraufnahmen aus dem Internet, die von leicht erkennbar bis schwer erkennbar reichten. Die Predictergebnisse sind ähnlich zu der Gesamttrefferquote des Modells und liegen im Bereich von 65-75 % und erfüllen unsere Erwartungen. Trotz schlechter Bildqualität können einige Bilder gut erkannt werden und das Modell ist robust gegen schlechte Bildbedingungen.



Abb. 18: Kohlmeise



Abb. 19: Feldsperling

Abb. 20: Zwei Beispiele für eigene Bilder.

6 Zusammenfassung

Unsere erzielten Ergebnisse haben voll und ganz unseren Erwartungen entsprochen. Mit einer Genauigkeit von rund 75 % haben wir ein Modell entwickelt, das zuverlässig Vögel klassifizieren kann. Die Ergebnisse zeigen, dass die Nutzung von Transfer Learning (VGG16) und Data Augmentation zu hohen Trefferquoten bei ähnlichen Vogelarten führt. Besonders gut werden Vogelarten mit ähnlichen Farb- und Formmerkmalen erkannt, während es bei visuell leichter zu unterscheidenden Arten häufiger zu Fehlklassifikationen kam. Eine kritische Analyse zeigt, dass die Bildqualität einen erheblichen Einfluss auf die Ergebnisse hatte. Besonders deutlich wurde dies bei Klassen mit hoher Varianz in der Bildqualität, da diese niedrigere Erkennungsraten erzielten. Dies legt nahe, dass der Datensatz optimiert werden kann und durch die gezielte Ergänzung qualitativ hochwertiger Bilder eine insgesamt bessere Modellleistung erzielt werden könnte. Aber trotzdem legt unsere Arbeit eine solide Basis für zukünftige Forschung. Zum einen könnten weitere Technologien zur Vermeidung von Überanpassung

integriert werden, um die Robustheit und Generalisierungsfähigkeit des Systems zu verbessern. Zum anderen bietet sich die Möglichkeit, unsere Ansätze auf andere Datensätze oder Problemstellungen zu übertragen, um zu untersuchen, inwieweit unsere Erkenntnisse dort anwendbar sind. Zudem könnte die Verwendung alternativer Architekturen eine zusätzliche Leistungssteigerung ermöglichen.